EE1007
Introduction to Computer Science Laboratory

Final Project
Design synthesis

Andy, Yu-Guang Chen
Assistant Professor, Department of EE
National Central University
andyygchen@ee.ncu.edu.tw

Background
Design procedure

- Design procedures or methodologies specify hardware that will implement a desired behavior.
- The design effort for small circuits may be manual, but the industry relies on automated synthesis tools for designing massive integrated circuits.
- Although it can meet our needs quickly and conveniently, but the cost of these Electronic Design Automation (EDA) tools license is too expensive.
- Therefore, many companies are now committed to the development of their own synthesis tools to reduce the cost.

Background
Design procedure

- Accordingly, NCU wants to develop their own synthesis tool that can automatically generate gate-level circuits from design descriptions.
- Since this is a massive project, NCU has separated the whole project into different small portions.
- One team has accomplished the transformation from design description to a state table, and the other team has implemented an algorithm to minimize complex logical expressions by Karnaugh map (K-map) minimization method.
- Your mission is to integrate these teams' programs and complete the synthesis tool which can automatically produce the corresponding circuit design based on the given design descriptions.

Description
Flow diagram

- The flow diagram of the NCU synthesis tool is shown in Fig. 1.

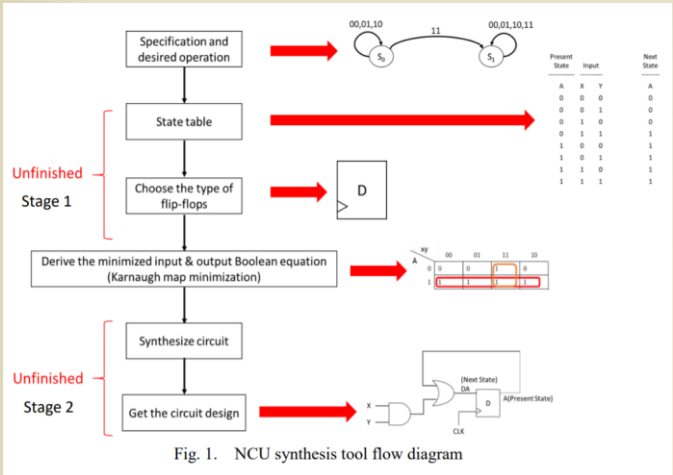


Fig. 1. NCU synthesis tool flow diagram

Description
Stage 1

- **Input:** The input of stage 1 is a state table. An example of a state table is shown in Fig. 2. To easily read this table into your program, the state table will be given in the input file named **"state_table.txt",** which includes the information of Fig.2. Note that we assume all Flip-flops are DFF to simplify the problem.

Present State	Input		Next State
A	X	Y	A
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	1
1	0	1	1
1	1	0	1
1	1	1	1

Fig. 2. State table

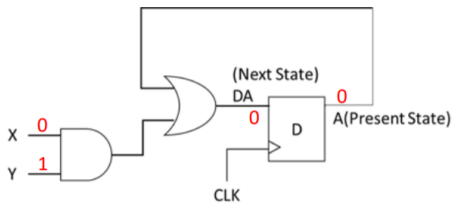


Fig. 3. Circuit design

Description
Stage 1

- An example is shown below:

```
Flip-Flop: D
//Present(A)
0
0
0
0
1
1
1
1
```

```
//Input(x,y)
0 0
0 1
1 0
1 1
0 0
0 1
1 0
1 1

//Next(A)
0
0
0
1
1
1
1
1

//Output(0)
```

Description
Stage 1

- Note that if there is a “0” in the parentheses, it means that the circuit doesn’t exist the corresponding node.
- In the example above, it shows this circuit has no output because there is a “0” in the output bracket.



Description
Stage 1

- **Output:** Your program should take the input file and generate a corresponding Karnaugh map stored in the file named **”kmap_in.txt”** which is used for the Karnaugh map minimization program developed by the other teams.
- An output file example is shown as follows. The details will be described in the next section.

```
1
3
0 0 1 0 1 1 1 1
```



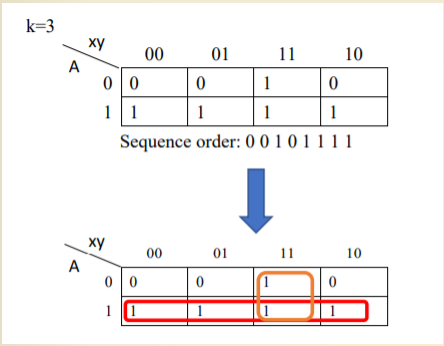
Description
Karnaugh Map Program User Guide

- The first line in the input file is an integer indicates the number of Boolean functions. Then each Karnaugh map will be represented by two lines.
- The first line is an integer to show the number of variables in the Boolean function. The second line represents the Boolean function in the Karnaugh map format.
- In the second line, the integers are corresponding to the value put in the squares of the Karnaugh map from top to down and left to right order. For each block in the Karnaugh map, the value can be either 1 for logic 1 and 0 for logic 0.

```
1       //number of Boolean functions
3       //number of Boolean functions variables
0 0 1 0 1 1 1 1
```


Description
Karnaugh Map Program User Guide

- The K-map minimization program will then perform logic reduction based on the rule mentioned in logic design class. In this example, the K-map minimization program will perform the reduction as follows:



Description
Karnaugh Map Program User Guide

- Therefore, the user can obtain the minimized Boolean function (SOP form) as "xy + A". Finally, K-map minimization program will generate an output file named **"kmap_out.txt"** and it represents the SOP form in the first 3 lines of the output file as follows where the first line gives the number label of the Boolean function.
- The value in the Boolean function can be 0, 1, or 2 (don't care).

```
#1          //The first function with the order (A, x, y)
2 1 1       // The term xy
1 2 2       // The term A
```


Description
Karnaugh Map Program User Guide

- To integrate this program into your code, you need to include their source code named **"kmap.cpp",** which we will provide to you, and use the function called void top() in your code.
- This function will automatically produce and store the minimized result in the file named **"kmap_out.txt".**

Description
Stage 2

- **Input:** After previous step, according to the Fig. 1, your program should take the result file named **"kmap_out.txt"** and process it.
- An input file example is shown follow:

```
#1
2 1 1
1 2 2
```


Description
Stage 2

- **Output:** After taking the input file, your program should produce a corresponding circuit. Notice that NCU has requested that this synthesis tool has to describe the circuit in benchmark format.
- An example is shown follow:

```
# 2 inputs
# 0 output
# 0 inverter
# 1 flip-flop
# 2 gates (1 AND + 1 OR)

INPUT(x)
INPUT(y)

N1 = AND(x,y)
DA = OR(A,N1)
A = D(DA,CLK)
#END
```

Description
Stage 2

- The first 5 lines starting with # are the comment lines. These comment lines provide basic information of the benchmark.
- Then a blank line appears. After that, all the input ports are specified one per line. Then a blank line appears again, follows by all the output ports. A blank line appears again, follows by circuit connection information.
- All interconnect wires in the circuit are named as “Nx” (x: 1~n) and all the output nodes of inverter are named as “!m” (m: the input node’s name of the inverter).
- Notice that if the circuit doesn’t exist input or output port, you don’t have to print that port. We will verify your program by comparing the circuit produced by your program with our solution.

Read Command Line

- In C++, main function usually don’t need to declare parameter for main. An example is shown below.

```
#include <stdio.h>
int main() {
  return 0;
}
```

Read Command Line

- If the program want to read input from command entered by the user, you can declare two parameters argc, argv in the main function. An example is shown below.

```
#include <iostream>
int main(int argc, char *argv[]) {
  std::cout << "We have " << argc << " arguments" << std::endl;
  for (int i = 0; i < argc; ++i) {
    std::cout << "[" << i << "] " << argv[i] << std::endl;
  }
  return 0;
}
```


Read Command Line

- **int argc** will store how many string you input from command line. char pointer array (string array) **char *argv[]** will store every string from command line in the **argv** array.

Read Command Line

- An example is shown below. After compiling source code, you will get a executive file named **a.out**.
- Compile **source.cpp**

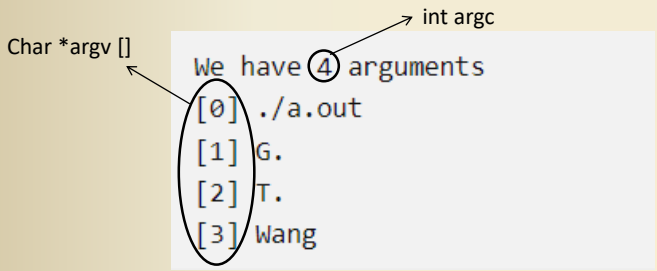
```
g++ source.cpp
```

- Execute **a.out**

```
./a.out G. T. Wang
```


Read Command Line

- After executing **a.out**, main function will take 4 arguments from the command line and store in the **argv** array. An example is shown below.





Reference



- argc, argv: https://blog.gtwang.org/programming/c-cpp-tutorial-argc-argv-read-command-line-arguments/

